COP 4710 Project Report: Case Vault

By Lily Sakuma, Kathrine Lu, Mia LaRocca, and Brandon Mealey

Our application is a cybersecurity vulnerability tracking system that uses data from CISA's Known Exploited Vulnerabilities (KEV) Catalog. The goal of the system is to store and organize information about vulnerabilities that are actively being exploited so users can easily view and analyze them. The application will allow users to search vulnerabilities, view details like the CVE ID and affected products, and connect them to investigation cases and evidence.

We chose this application because cybersecurity vulnerabilities are a real and critical issue today. Instead of using completely made-up data, we wanted to work with real data from the web provided by CISA. The main parts of our project include users, cases, and evidence. Users can create and manage cases, which can contain evidence, and vulnerabilities from the KEV catalog can be linked to those cases. This allows the system to organize cybersecurity information in a structured way using a relational database.

Database Details:

Case Vault is a cyber incident reporting and case management system backed by a relational database. The database was implemented in SQLite3 and is designed to store, organize, and efficiently query cybersecurity incident data for small organizations. The design of the database prioritizes structural clarity, referential integrity, and normalization to eliminate redundancy and support the system's core workflows which are: creating incidents, assigning handlers, linking evidence and assets, and generating analytical reports.

Entity-Relationship (ER) Model

The E-R model defines four primary entities, each with a distinctly identified primary key and a set of descriptive attributes:

Entity	Primary Key	Attributes
User	user_id	first_name, last_name, email
Case	case_id	title, description, severity, status, opened_date, created_by (FK -> User)
Evidence	evidence_id	case_id (FK -> Case), type, description, acquired_date
Asset	asset_id	asset_name, asset_type

In the original ER diagram, the 'manages' and 'handles' relationships between User and Evidence/Case were represented as diamond-shaped relationship sets. In the updated schema

(Stage 3), these many-to-many relationships were correctly converted into junction tables (Case_Handler, Case_Asset) so that they can be represented in a flat relational model without introducing multi-valued columns.

Relationships

The following relationships connect the entities:

Relationship	Cardinality	Description
User creates Case	1 : N	A user may report many cases; each case has exactly one creator (creater_by)
User handles Case	M : N	Multiple analysts can handle multiple cases, captured by the Case_Handler join table
Case contains Evidence	1 : N	One case can have many evidence items; each evidence item belongs to one case.
Case impacts Asset	M : N	One case can involve many assets; one asset can be involved in many cases, captured by the Case_Asset join table

Relational Schema

The following tables constitute the full relational schema of Case Vault. Foreign Key references enforce referential integrity across tables.

User - User(user_id, first_name, last_name, email)

Case - Case(case_id, title, description, severity, status, opened_date, created_by)

Evidence - Evidence(evidence_id, case_id, type, description, acquired_date)

Asset - Asset(asset_id, asset_name, asset_type)

Case_Asset (Junction Table) - Case_Asset(case_id, asset_id)

Case_Handler (Junction Table) - Case_Handler(case_id, user_id)

Functional Dependencies

The following functional dependencies (FDs) were identified for each table:

User – user_id -> first_name, last_name, email

Case – case_id -> title, description, severity, status, opened_date, created_by

Evidence – evidence_id -> case_id, type, description, acquired_date

Asset – asset_id -> asset_name, asset_type

Case_Asset – (case_id, asset_id) -> (no non-key attributes); PK is the entire row

Case_Handler - (case_id, user_id) -> (no non-key attributes); PK is the entire row

Normal Form Analysis

First Normal Form (1NF)

All tables satisfy 1NF. Every column holds a single atomic value – there are no repeating groups or set-valued attributes. For example, the many-to-many relationships between cases and assets (and cases and handlers) that could have been stored as comma-separated lists are instead factored into separate junction tables (Case_Asset, Case_Handler), preserving atomicity.

Second Normal Form (2NF)

All tables satisfy 2NF. 2NF requires that every non-key attribute be fully functionally dependent on the entire primary key (no partial dependencies). For the tables with single-column primary keys (User, Case, Evidence, Asset), this is satisfied trivially. For the junction tables Case_Asset and Case_Handler, the primary key is the composite of both foreign key columns, and there are no non-key attributes at all – so there can be no partial dependency.

Third Normal Form (3NF) and BCNF

All tables satisfy 3NF and are in Boyce-Codd Normal Form (BCNF). 3NF requires that no non-key attribute is transitively dependent on the primary key through another non-key attribute. BCNF strengthens this by requiring that every non-trivial functional dependency $X \rightarrow Y$ has X as a superkey.

Examining each table:

- User: user_id is the sole determinant. email is UNIQUE but all non-key attributes depend only on user_id. No transitive dependencies exist.
- Case: case_id is the sole determinant. created_by is a foreign key reference but does not create a transitive dependency — it is an attribute of a Case, not a derived fact about another attribute of Case.

- Evidence: evidence_id determines all other attributes. case_id is a foreign key but again does not create a transitive dependency within the Evidence table itself.
- Asset: asset_id determines asset_name and asset_type. No transitive dependencies.
- Case_Asset / Case_Handler: These tables have no non-key attributes, so 3NF and BCNF are satisfied trivially — the only FD is the trivial one from the composite PK to itself.

Therefore, all six tables are in BCNF, which implies they are also in 3NF, 2NF, and 1NF.

Summary

The Case Vault database is built on a clean, normalized relational schema consisting of six tables. The design eliminates data redundancy through BCNF normalization: all non-key attributes depend directly and solely on their table's primary key, with no partial or transitive dependencies. Many-to-many relationships are correctly resolved into junction tables. Integrity is enforced through primary keys, foreign keys with cascade behavior, NOT NULL constraints, UNIQUE constraints, and domain-level CHECK constraints. This foundation supports the system's advanced querying requirements, including multi-table joins for incident context, aggregate statistics, and correlation queries, all while maintaining data quality and consistency throughout the incident lifecycle.

Functionality details:

The core functionality of our system includes inserting, retrieving, updating, and deleting records within the database. Record insertion is implemented through SQL INSERT statements executed in Python using the SQLite3 library. Parameterized queries are used to ensure secure data handling and prevent SQL injection vulnerabilities. For instance, when a new user or case is created, the system captures user input and inserts it into the appropriate table while preserving data integrity. Data retrieval is performed using SELECT queries, enabling users to search for records based on specific attributes such as case status or severity. The results are then formatted and displayed in a structured manner to facilitate efficient browsing and analysis.

In addition to basic operations, the system supports more advanced queries involving joins and aggregation. Join operations allow data from multiple tables to be combined, such as linking cases with the users who created or are assigned to them. This enables the system to present more meaningful and relational data rather than isolated records. Aggregation is implemented using SQL functions such as COUNT and GROUP BY, allowing the system to generate summaries, such as the number of cases by severity level. Updating records is handled through UPDATE statements, enabling changes like changing a case’s status. Similarly, deletion is performed using DELETE statements, allowing records to be removed while maintaining relational consistency through enforced constraints.

Beyond these core functions, the system includes advanced features that enhance usability and demonstrate more complex functionality. One such feature is multi-criteria search,

which allows users to filter results based on multiple conditions simultaneously, such as searching by both case status and severity. This is implemented by dynamically constructing SQL queries in Python based on the user's selected filters. Another advanced capability is the use of analytical queries to generate insights, such as determining the number of cases assigned to each user or identifying trends in case severity. These features leverage more complex SQL operations and improve the system's ability to support data-driven analysis.

The system also supports relationship-based data retrieval, where related information from multiple tables is accessed using join operations. For example, users can retrieve a case along with its associated evidence, requiring the combination of data across tables through foreign key relationships. Data integrity is enforced using primary and foreign keys defined within the schema, ensuring consistency and preventing invalid data entry. Additionally, error handling is implemented in Python using try-except blocks to manage potential issues during database operations. Transactions are used to ensure atomicity, guaranteeing that operations are either fully completed or rolled back in the event of an error, thereby maintaining database reliability.

Overall, the system satisfies all required core functionalities while incorporating advanced features that enhance performance, flexibility, and usability. By integrating SQLite with Python, the system demonstrates a practical and robust approach to developing a database-driven application capable of supporting both standard operations and complex data interactions.

Implementation details:

For the implementation of our project, we selected a combination of Python, Flask, HTML, Cascading Style Sheets (CSS), and SQLite as our primary technologies. These tools were chosen based on their accessibility, compatibility with one another, and suitability for our project guidelines that still demonstrate core database and web application concepts. Our goal was to build a system that clearly separates the front-end interface, application logic, and database management, while still being simple enough to complete within the project timeline.

Python was chosen as the main programming language due to its readability, flexibility, and strong support for database interaction. It is widely used in both industry and schools, making it an easy choice for developing application logic. Additionally, Python integrates well with SQLite and Flask, allowing us to build a full-stack application without needing multiple complex environments. Flask is a lightweight web framework for Python, so it was selected to handle routing, user request, and communication between the front-end and the database. Unlike larger frameworks, Flask provides only the most pivotal tools needed, which made it easier for us to understand and control each part of the application.

For the database system, we chose SQLite because it is a serverless, file-based relational database management system. This means that the entire database is stored in a single file which eliminated the need for installation of configuration of a separate database server like MySQL or PostgreSQL. This simplicity made SQLite ideal for our project since it allowed us to focus on

database design and query implementation rather than setup and maintenance. Even with its simplicity, SQLite still supports standard SQL operations like SELECT, INSERT, UPDATE, DELETE, and JOIN, which are important for demonstrating relational database concepts.

The front-end web interface of our application is implemented using HTML and CSS. HTML is used to define the structure of each page, including forms, tables, and navigation elements. CSS is used to improve the visual presentation and usability of the interface, allowing easier access for users to interact with the system. Multiple pages are included in the front-end that allow users to perform different operations, like viewing records, adding new data, updating existing entries, and deleting records. A key component of the interface is the input forms, as they allow users to submit information that will be processed by the application.

To strengthen usability, we created the interface to be straightforward and intuitive. Each page is structured so that users can clearly understand what action they are performing. For example, pages that display data use tables with labeled columns, while pages that collect input use clearly labeled form fields. Buttons such as “Submit”, “Delete”, and even “Update” are placed in logical locations to guide user interaction. Although we kept the visual design simple, we wanted the focus on functionality and clarity rather than advanced styling.

The application logic is implemented using Python within the Flask framework. This layer acts as the intermediary between the user interface and the database. For example, when a user submits a form or interacts with the front-end in any way, the request is sent to the Flask application. Flask then processes the request by extracting the user's input, validating it, and determining which database operation needs to be performed. This may involve interesting new data, updating existing records, deleting entries, or retrieving information from the database.

Input validation is an important part of the application logic. Before any data is sent to the database, the application checks that the input is complete and in the correct format. This helps prevent errors and ensures that the database remains consistent. For example, required fields must be filled in, and data types must match the expected format (such as numeric values for IDs or text values for names). By handling validation at the application level, we reduce the likelihood of invalid data being stored in the database.

Once the input has been validated, the application executes SQL quires using Python’s built-in SQLite library. These queries are written using standard SQL syntax and are dynamically generated based on user actions. For example, when a user submits a form to add a new record, the application constructs an INSERT statement and executes it against the database. Similarly, when a user searches for data, the application generates a SELECT query with appropriate conditions to retrieve the relevant records.

The interaction between the front-end and the backend database is handled entirely through the Flask application. The front-end does not communicate directly with SQLite. Instead, it sends HTTP requests to the Flask server whenever the user performs an action. Flask receives these requests, processes them, and interacts with the database as needed. After executing the appropriate SQL query, Flask retrieves the results and passes them to an HTML template. These templates are then rendered into web pages that display the results to the user.

The architecture provides a clear separation of responsibilities. The front-end is responsible for presenting information and collecting user input; the application logic handles processing and decision-making, and the database is responsible for storing and managing data. This separation improves the system's maintainability and scalability, as each component can be modified independently without affecting the others.

To manage the dynamic content on the front-end, we used Flask's templating system. Templates allow us to embed Python variables and logic directly into HTML files. For example, when displaying a list of records, the application passes the data retrieved from the database to the template, which then loops through the data and generates a table dynamically. This approach makes it easier to display updated information without manually modifying the HTML each time.

In terms of data flow, the process begins when a user interacts with the interface, such as submitting a form of clicking a button. This action sends a request to the Flask application, which processes the request and determines the appropriate SQL query. The query is executed on the SQLite database, and the results are returned to the application. The application then formats the results and sends them back to the front-end, where they are displayed to the user. This cycle repeats each user's interaction, allowing the system to function as a fully interactive web application.

For development and testing, we used tools such as DB browser for SQLite to create tables, inspect data, and test SQL queries before integrating them into the application. This helped ensure that our queries were correct and efficient before being used in the code. By testing queries separately, we were able to identify and fix errors early in the development process. To support collaboration and version control, we decided to store the project code in an online repository called GitHub.

Overall, the combination of Python, Flask, HTML, CSS, and SQLite provides a complete and functional system for implementing a database-driven web application. This approach allows us to demonstrate key concepts such as relational database design, SQL query execution, user interaction, and the integration of multiple technologies into a cohesive system, while the system is relatively simple, it effectively illustrates the core principles required to build and manage a database-backed web application.

Experience

Throughout this project, we learned how to design and implement a complete database-driven application by connecting the database, application logic, and user interface. One of the most important key takeaways was understanding how these components work together. We also gained experience with database design, including creating tables, defining primary keys, and organizing data efficiently to avoid redundancy.

We improved our skills in writing and testing SQL queries such as SELECT, INSERT, UPDATE, and DELETE. A key lesson was learning how to debug queries by testing them step by step and verifying results before changing the database. This helped us avoid errors and better understand how queries interact with stored data.

One of the main challenges we faced was ensuring the data flowed correctly from the front-end to the database. At times, user input was not stored or retrieved properly. We solved this by debugging each step of the process, starting from the form input, then checking the application logic in Flask, and finally verifying the SQL queries. Breaking the problem into smaller parts made it easier for us to identify and fix issues. Another challenge was preventing incorrect or incomplete data from being entered into the database. To address this, we added validation checks in the application logic to ensure that inputs were in the correct format and required fields were filled. This improved the reliability and consistency of the system.

In the future, this project could be extended by using a more advanced database system such as MySQL or PostgreSQL to support larger datasets and multiple users. The front-end could also be improved by using modern frameworks like React to create a more dynamic and interactive user experience. Additional features such as user authentication, role-based access, and integration with external data sources could make the system more realistic and secure. Finally, deploying the application online would allow it to function as a fully accessible web system.

This project helped us develop a better understanding of database systems, application development, and problem solving. It also gave us practical experience in debugging and building a functional system from start to finish.

References

Cybersecurity and Infrastructure Security Agency. (2023). *Known Exploited Vulnerabilities Catalog | CISA*. Www.cisa.gov. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

OpenAI. (2026). *ChatGPT* (April 24 version) [Large language model]. https://chat.openai.com/

Python. (2020). *The Python Standard Library — Python 3.8.1 documentation*. Python.org. https://docs.python.org/3/library/index.html

SQLite Home Page. (2019). Sqlite.org. https://sqlite.org/

W3Schools. (2024). *HTML Tutorial*. W3schools.com. https://www.w3schools.com/html/